



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS



PROGRAMACIÓN

ASIGNATURA:

PROGRAMACIÓN

PROFESOR:

Ing. Yadira Franco Rocha

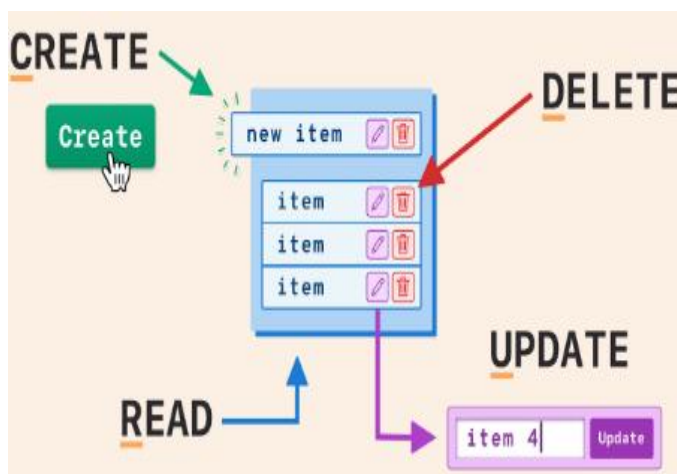
PERÍODO ACADÉMICO:

2025-B

PROYECTO

TÍTULO

PROYECTO SEGUNDO BIMESTRE



ESTUDIANTES:

Lizbeth Sánchez Andrimba

Índice

1. INTRODUCCIÓN.....	3
1.1. Propósito del documento	3
2. OBJETIVOS	3
2.1. Objetivo General	3
2.2. Objetivos Específicos.....	3
3. DISEÑO DE LOS COMPONENTES.....	3
3.1. Interfaces implementadas	3
3.2. Diálogos secundarios (QDialog):.....	5
4. DESCRIPCIÓN DE LAS FUNCIONES	6
5. ESTRUCTURA DEL SISTEMA	6
5.1. Estructura funcional del sistema	7
6. ESTRUCTURA DEL PROYECTO	7
6.1 Carpeta Proyecto_productos.....	8
6.2 Carpeta Forms.....	8
6.3 Carpeta Header Files	8
6.4 Carpeta Source Files	8
7. CONCLUSIONES	9
8. RECOMENDACIONES	9
9. REFERENCIAS	9

1. INTRODUCCIÓN

En el desarrollo de sistemas informáticos, la seguridad y el control de acceso son aspectos fundamentales para garantizar un uso adecuado de la información. Un sistema de login permite autenticar a los usuarios antes de que puedan acceder a las funcionalidades de un sistema, evitando accesos no autorizados.

1.1. Propósito del documento

El propósito del presente informe es describir el desarrollo, diseño y funcionamiento del sistema CRUD, detallando sus objetivos, componentes, funcionalidades y estructura general, así como los resultados obtenidos durante su implementación.

2. OBJETIVOS

2.1. Objetivo General

Desarrollar un sistema de registro de información con interfaz gráfica que permita autenticar usuarios de forma segura y eficiente, utilizando el lenguaje de programación C++ y el framework Qt.

2.2. Objetivos Específicos

- Diseñar una interfaz gráfica intuitiva y amigable para el usuario.
- Garantizar el correcto funcionamiento del sistema mediante pruebas básicas.
- Implementar validaciones para los campos de entrada.

3. DISEÑO DE LOS COMPONENTES

El sistema fue diseñado utilizando interfaces gráficas que permiten una interacción clara y sencilla con el usuario. Las interfaces desarrolladas fueron las siguientes:

3.1. Interfaces implementadas

- **Interfaz de Login:** permite al usuario ingresar su nombre de usuario y contraseña.



Dialog

✦ **BIENVENIDOS AL SISTEMA** ✦

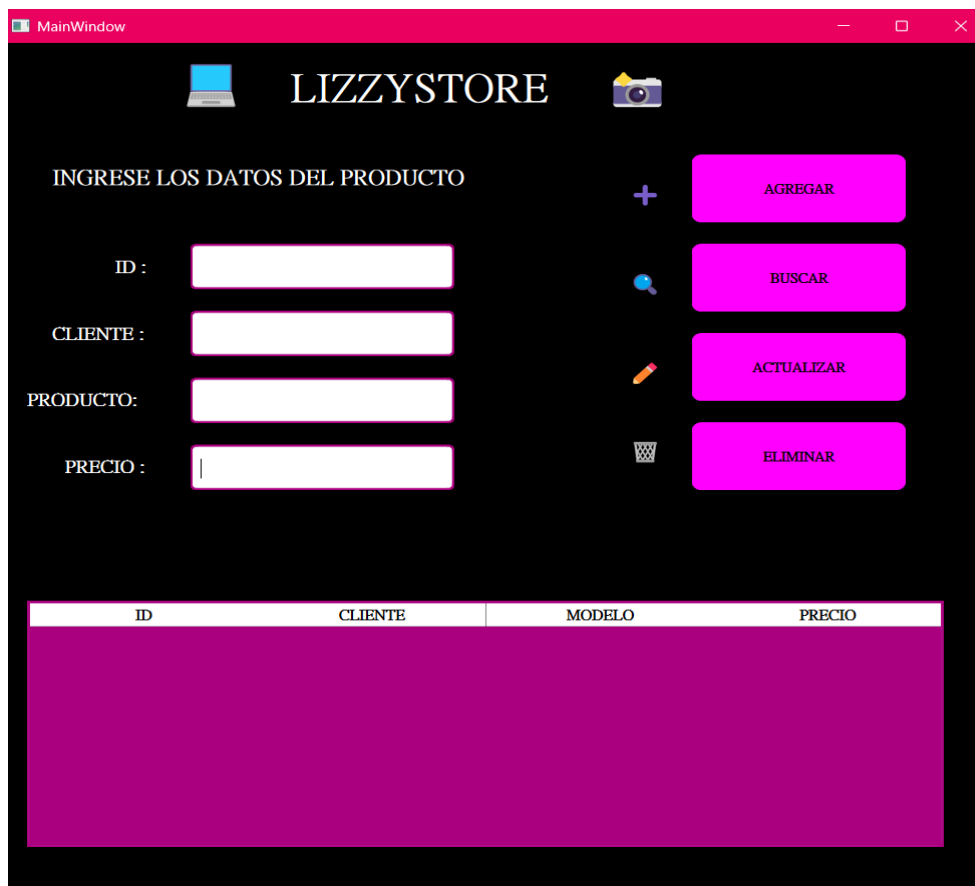
LizzyStore

USUARIO

CONTRASEÑA

INGRESAR

- **Interfaz de Registro de información:** permite el registro de nueva información en el sistema.



MainWindow

LIZZYSTORE

INGRESE LOS DATOS DEL PRODUCTO

ID :

CLIENTE :

PRODUCTO:

PRECIO :

AGREGAR

BUSCAR

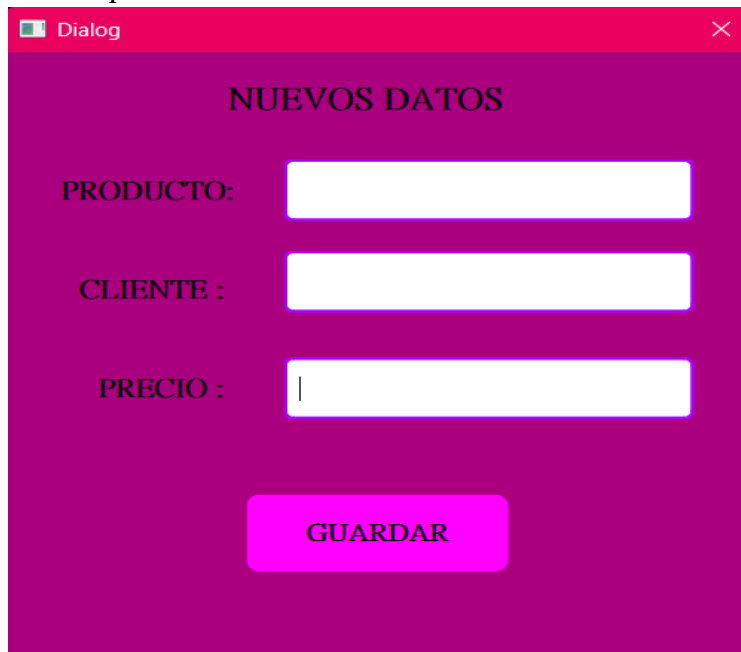
ACTUALIZAR

ELIMINAR

ID	CLIENTE	MODELO	PRECIO
----	---------	--------	--------

3.2. Diálogos secundarios (QDialog):

- **Diálogo de actualización:** se abre una ventana emergente que permite ingresar los nuevos datos del producto a actualizar.



Dialog

NUEVOS DATOS

PRODUCTO:

CLIENTE :

PRECIO :

GUARDAR

- **Diálogo de búsqueda:** muestra únicamente el producto que fue buscado, facilitando su visualización.



Datos del Producto

ID

CLIENTE

PRODUCTO

PRECIO

CERRAR

3.3. Funcionalidades implementadas

- Ingreso de usuario y contraseña.
- Ocultamiento visual de la contraseña mediante caracteres especiales.
- Validación de campos vacíos.
- Mensajes de error y confirmación.
- Acceso o denegación según las credenciales ingresadas.
- Apertura de diálogos para actualizar información.
- Visualización de datos específicos según la acción realizada.

4. DESCRIPCIÓN DE LAS FUNCIONES

El sistema cuenta con diversas funciones que permiten su correcto funcionamiento, entre las cuales se destacan:

1. **CRUD:** Crear, Leer, Actualizar, Eliminar.
2. **Funciones Principales:**
 - crearProducto()
 - mostrarProducto()
 - buscarProducto()
 - actualizarProducto()
 - eliminarProducto()
 - main()
3. **Archivos de datos:** Guardar los productos en una base de datos.
4. **Struct Producto:** Manejar el registro de los productos (ID, nombre, producto, precio)
5. **Función de validación:** verifica que los campos no estén vacíos antes de continuar.
6. **Función de autenticación:** compara los datos ingresados con los datos registrados.
7. **Función de mensajes:** muestra alertas informativas o de error al usuario.
8. **Interfaz Gráfica:** Visualizar el programa mediante imagen.

Estas funciones trabajan de manera conjunta para garantizar una experiencia de usuario segura y ordenada.

5. ESTRUCTURA DEL SISTEMA

El sistema está estructurado de forma sencilla, facilitando su comprensión y mantenimiento.

Se organiza en los siguientes componentes:

- **Interfaz gráfica (UI):** diseñada con Qt Designer.
- **Lógica del sistema:** implementada en archivos .cpp, donde se realizan las validaciones y procesos.

Esta estructura permite separar la lógica del diseño visual, mejorando la organización del código.

5.1. Estructura funcional del sistema

Crear Producto: Esta función tiene como finalidad insertar o registrar nuevos datos al sistema, almacenando la información de forma organizada en una base de datos o archivos txt la cual es una herramienta que nos servirá para manipular la información registrada las veces que sean necesarias.

Mostrar Producto: Consultar o mostrar los datos existentes del sistema, ya sea un solo registro o toda la lista completa, permitiéndonos observar toda la información almacenada en el archivo.

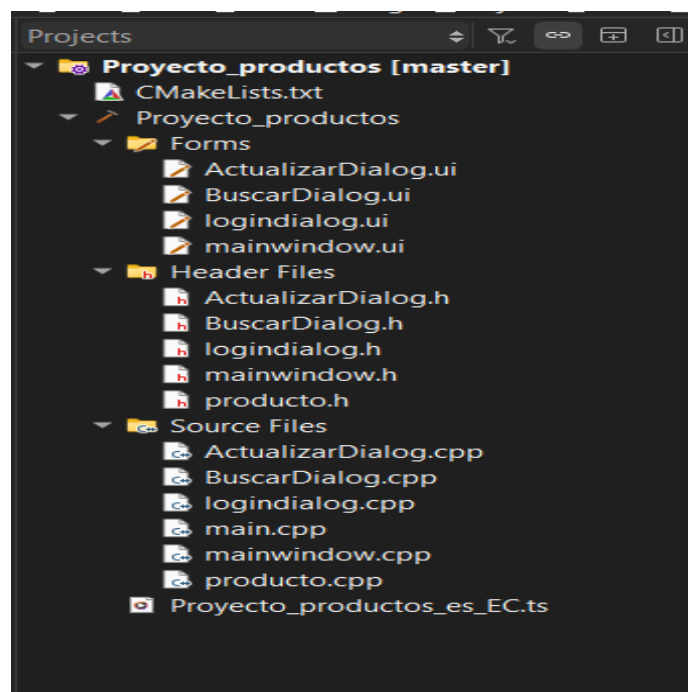
Buscar Producto: Función que nos ayuda a buscar un único registro en específico mediante su ID de identificación. Listando solo la información del registro a buscar.

Actualizar Producto: En esta función nos permite modificar o actualizar cualquier información del archivo, sin afectar al resto del registro únicamente se modifica el registro que nosotros deseamos.

Eliminar Producto: Elimina los registros del sistema, de igual manera se elimina el registro que nosotros deseamos sin afectar a la lista completa de los datos.

Archivos: El programa guarda la información registrada en un archivo de texto llamados productos.txt, donde se usa ofstream para guardar y ifstream para cargar.

6. ESTRUCTURA DEL PROYECTO



6.1 Carpeta Proyecto_productos

Esta carpeta es la principal, ya que dentro de ella contiene todas las funcionalidades del proyecto, estructura, interfaces, código fuente y archivos.

6.2 Carpeta Forms

Dentro de ella contiene los archivos .ui los cuales son creados con QT Designer donde se puede encontrar el diseño visual de cada una de las interfaces.

- **loginDialog.ui:** Interfaz visual del login.
- **mainwindow.ui:** Interfaz principal donde se registrarán los datos de cada producto.
- **ActualizarDialog.ui:** Interfaz adicional donde el usuario ingresara los nuevos datos de un producto.
- **BuscarDialog.ui:** Interfaz adicional que nos permitirá visualizar un solo registro del sistema.

6.3 Carpeta Header Files

Los archivos .h se utilizan para declarar la estructura de una clase, pero no su lógica completa.

- **ActualizarDialog.h:** Este archivo contiene la declaración del dialog encargado de actualizar un producto.
- **BuscarDialog.h:** Este archivo contiene la declaración del dialog utilizado para mostrar el resultado de una búsqueda.
- **logindialog.h:** Este archivo contiene la declaración del dialog de autenticación del sistema.
- **mainwindow.h:** Este archivo contiene la declaración de la ventana principal del sistema.
- **producto.h:** Este archivo contiene la declaración de la clase Producto, que representa la estructura de los datos.

6.4 Carpeta Source Files

Esta carpeta contiene los archivos con extensión .cpp, los cuales incluyen la implementación de la lógica del sistema. Donde nos dice cómo funciona el sistema.

- **ActualizarDialog.cpp:** Este archivo contiene la implementación lógica del dialog actualizar, donde se cargan y se registran los nuevos datos ingresados.
- **BuscarDialog.cpp:** Este archivo implementa la lógica del dialog de búsqueda, donde incluye la recepción del producto buscado y la visualización de los datos.
- **logindialog.cpp:** Este archivo contiene la lógica para el ingreso al sistema, validando el usuario y la contraseña, de tal manera que permite o niega el acceso.
- **main.cpp:** Este archivo es el punto de inicio de la aplicación. Contiene la función main(), la creación del objeto QApplication y su ejecución, en si inicia y controla el ciclo de vida del programa.
- **mainwindow.cpp:** Este archivo implementa la lógica de la ventana principal. Controla las operaciones principales del sistema después del login.
- **producto.cpp:** Este archivo contiene la implementación de la clase Producto.

7. CONCLUSIONES

1. Se logró desarrollar un sistema de login funcional y estable.
2. El uso de Qt permitió crear una interfaz gráfica amigable.
3. Se aplicaron conceptos fundamentales de programación y validación de datos.
4. El proyecto fortaleció los conocimientos adquiridos en la asignatura.

8. RECOMENDACIONES

1. Implementar cifrado de contraseñas para mayor seguridad.
2. Integrar una base de datos para el almacenamiento de usuarios.
3. Añadir roles de usuario con distintos permisos.
4. Mejorar el diseño visual para una experiencia más profesional.

9. REFERENCIAS

- ProgramacionDylan. QT C++ 2024 - Tu primera aplicación con interfaz gráfica YouTube. <https://youtu.be/JDpvtcE436Y>
- Programacion Desde Cero. Cómo hacer un programa con interfaz gráfica en C++ y QT. YouTube. <https://youtu.be/cnFItSiEJTU>
- Marita moreno torres. *Como cambiar de color y tamaño las letras en QT creator. YouTube. <https://youtu.be/UxWRogCjS6Y>
- Jesús Conde. 0.4 Curso de PyQT Crear Interface de Usuario con QT. YouTube. <https://youtu.be/UDGpEqqjgXs>